

Técnicas de programación en WindowBuilder

Three horizontal bars of different shades of green and blue, stacked on top of each other, spanning the width of the slide.

Programación III - UNGS

Absolute layout

- Por defecto, las bibliotecas **Swing** ubican los controles en posiciones predeterminadas (norte, sur, centro, etc.).

Absolute layout

- Por defecto, las bibliotecas **Swing** ubican los controles en posiciones predeterminadas (norte, sur, centro, etc.).
- Para evitar esto, se debe ubicar un **JAbsoluteLayout** como primer componente del frame, para especificar coordenadas absolutas para los controles que se posicionen a continuación en el **designer**.

Look and feels

- Se pueden seleccionar distintas apariencias para nuestra aplicación en Java. El conjunto de características que determinan la apariencia de la aplicación se denomina **look and feel**.

Look and feels

- Se pueden seleccionar distintas apariencias para nuestra aplicación en Java. El conjunto de características que determinan la apariencia de la aplicación se denomina **look and feel**.
- Para usar el look and feel del sistema, se deben agregar estas líneas en el constructor del form principal:

Look and feels

- Se pueden seleccionar distintas apariencias para nuestra aplicación en Java. El conjunto de características que determinan la apariencia de la aplicación se denomina **look and feel**.
- Para usar el look and feel del sistema, se deben agregar estas líneas en el constructor del form principal:

```

1  try
2  {
3      UIManager.setLookAndFeel(
4          UIManager.getSystemLookAndFeelClassName());
5  }
6  catch(Exception e) { ... } // Debe estar en un try/catch
    
```

Look and feels

- Para usar un look and feel alternativo, es necesario instalar el archivo .jar correspondiente dentro del *build path* del proyecto y cambiar la referencia en el método **setLookAndFeel()**.

Look and feels

- Para usar un look and feel alternativo, es necesario instalar el archivo .jar correspondiente dentro del *build path* del proyecto y cambiar la referencia en el método **setLookAndFeel()**.
- Es importante utilizar un look and feel que se corresponda con las bibliotecas gráficas que estamos usando (en nuestro caso, usamos **Swing**).

Look and feels

- Para usar un look and feel alternativo, es necesario instalar el archivo .jar correspondiente dentro del *build path* del proyecto y cambiar la referencia en el método **setLookAndFeel()**.
- Es importante utilizar un look and feel que se corresponda con las bibliotecas gráficas que estamos usando (en nuestro caso, usamos **Swing**).
- Por ejemplo, para usar el look and feel **JTattoo**, hacemos:

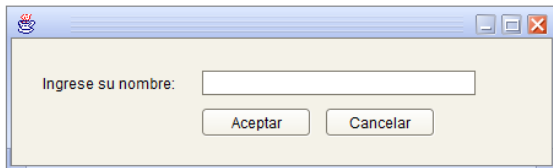
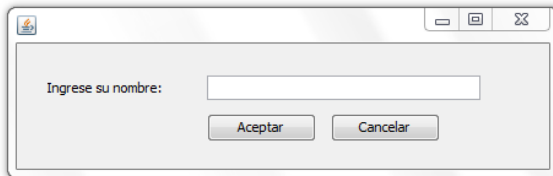
Look and feels

- Para usar un look and feel alternativo, es necesario instalar el archivo .jar correspondiente dentro del *build path* del proyecto y cambiar la referencia en el método **setLookAndFeel()**.
- Es importante utilizar un look and feel que se corresponda con las bibliotecas gráficas que estamos usando (en nuestro caso, usamos **Swing**).
- Por ejemplo, para usar el look and feel **JTattoo**, hacemos:

```

1  try
2  {
3      UIManager.setLookAndFeel("com.jtattoo.plaf.smart.SmartLookAndFeel");
4  }
5  catch(Exception e) { ... }
```

Look and feels



Text fields

- Los **JTextField**s permiten consultar y modificar el texto con **getText()** y **setText()**, respectivamente.

Text fields

- Los **JTextField**s permiten consultar y modificar el texto con **getText()** y **setText()**, respectivamente.
- Es importante recordar que estos dos métodos operan con **Strings**, con lo cual hay que hacer conversiones explícitas hacia/desde los otros tipos de datos:

Text fields

- Los **JTextField**s permiten consultar y modificar el texto con **getText()** y **setText()**, respectivamente.
- Es importante recordar que estos dos métodos operan con **Strings**, con lo cual hay que hacer conversiones explícitas hacia/desde los otros tipos de datos:

```

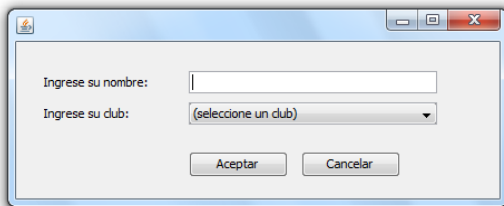
1  String str = textField.getText();
2  int valor = Integer.parseInt(str); // Puede fallar
3
4  valor = valor + 1;
5  textField.setText( Integer.toString(valor) );
```

Combo boxes

- Un **combo box** es adecuado cuando el usuario debe seleccionar entre un conjunto de opciones grande o bien determinado dinámicamente.

Combo boxes

- Un **combo box** es adecuado cuando el usuario debe seleccionar entre un conjunto de opciones grande o bien determinado dinámicamente.
- Se evita que el usuario seleccione una alternativa incorrecta.



A screenshot of a standard Windows-style dialog box. The dialog has a title bar with a small icon on the left and standard minimize, maximize, and close buttons on the right. The main area contains two labels: "Ingrese su nombre:" followed by a single-line text input field, and "Ingrese su club:" followed by a combo box. The combo box currently displays the text "(seleccione un club)" and has a small downward-pointing arrow on its right side. At the bottom of the dialog, there are two buttons: "Aceptar" (Accept) and "Cancelar" (Cancel).

Combo boxes

- Para cargar las opciones (*populate*) del combo, se debe generar un nuevo **DefaultComboBoxModel** con un arreglo de strings:

Combo boxes

- Para cargar las opciones (*populate*) del combo, se debe generar un nuevo **DefaultComboBoxModel** con un arreglo de strings:

```

1  comboBox.setModel(new DefaultComboBoxModel(new String[]
2      {"(seleccione un club)", "River Plate", "Boca Juniors",
3      "Independiente", "Racing Club", "Otros" }));
  
```

Combo boxes

- Para cargar las opciones (*populate*) del combo, se debe generar un nuevo **DefaultComboBoxModel** con un arreglo de strings:

```

1  comboBox.setModel(new DefaultComboBoxModel(new String[]
2      {"(seleccione un club)", "River Plate", "Boca Juniors",
3      "Independiente", "Racing Club", "Otros" }));
    
```

- El **ComboBoxModel** (y en general, los **models** para los componentes de Swing) es el **modelo de datos** del control, que especifica qué datos se muestran y cómo se seleccionan.

Combo boxes

- Para obtener el ítem seleccionado, se puede consultar por índice o por valor, teniendo en cuenta que en el segundo caso se obtiene un **Object**.

Combo boxes

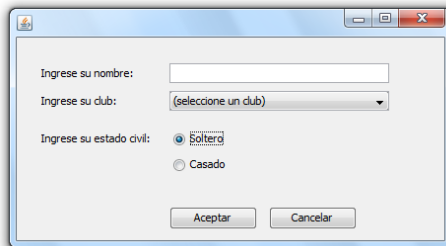
- Para obtener el ítem seleccionado, se puede consultar por índice o por valor, teniendo en cuenta que en el segundo caso se obtiene un **Object**.

```

1  int indice = comboBox.getSelectedIndex();
2  String valor = (String)comboBox.getSelectedItem();
    
```

Radio buttons

- Cuando las opciones son pocas y están prefijadas de antemano, se puede usar un grupo de **JRadioButtons**:



Ingrese su nombre:

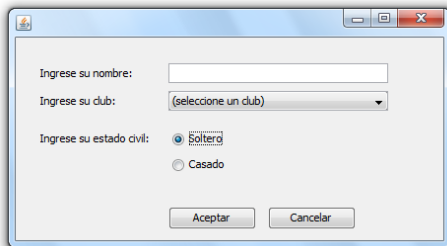
Ingrese su club: (seleccione un club) ▼

Ingrese su estado civil: ☒ Soltero ☐ Casado

Aceptar Cancelar

Radio buttons

- Cuando las opciones son pocas y están prefijadas de antemano, se puede usar un grupo de **JRadioButtons**:



- Se deben ubicar todos los radio buttons dentro de un mismo **ButtonGroup**. Para esto, seleccionar a todos los radio buttons → botón derecho → *Set button group* → *New standard* (o bien seleccionar un **ButtonGroup** existente).

Radio buttons

- Esto genera el siguiente código:

Radio buttons

- Esto genera el siguiente código:

```

1  private final ButtonGroup buttonGroup = new ButtonGroup();
2  ...
3  JRadioButton rdbtnSoltero = new JRadioButton(" Soltero" );
4  JRadioButton rdbtnCasado = new JRadioButton(" Casado" );
5  buttonGroup.add(rdbtnSoltero);
6  buttonGroup.add(rdbtnCasado);

```

Radio buttons

- Esto genera el siguiente código:

```

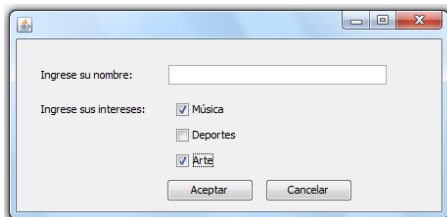
1  private final ButtonGroup buttonGroup = new ButtonGroup();
2  ...
3  JRadioButton rdbtnSoltero = new JRadioButton(" Soltero");
4  JRadioButton rdbtnCasado = new JRadioButton(" Casado");
5  buttonGroup.add(rdbtnSoltero);
6  buttonGroup.add(rdbtnCasado);

```

- Un **ButtonGroup** controla que haya a lo sumo un radio button seleccionado dentro del grupo. Si no se agrupan los radio buttons de esta forma, puede haber dos de ellos seleccionados al mismo tiempo.

Check boxes

- Cuando las opciones no son excluyentes, se prefieren los **JCheckBoxes**:



Ingrese su nombre:

Ingrese sus intereses:

☒ Música

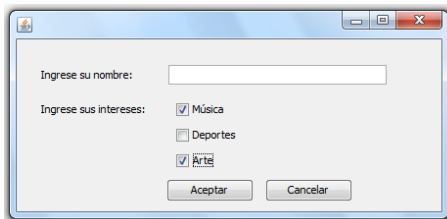
☐ Deportes

☒ Arte

Aceptar Cancelar

Check boxes

- Cuando las opciones no son excluyentes, se prefieren los **JCheckBoxes**:



- Para obtener si un check box está activado, se utiliza el método **boolean isSelected()** sobre el **JCheckBox**.

Tablas

- La visualización de conjuntos de datos se realiza habitualmente por medio de **JTables**:

Tablas

- La visualización de conjuntos de datos se realiza habitualmente por medio de **JTables**:

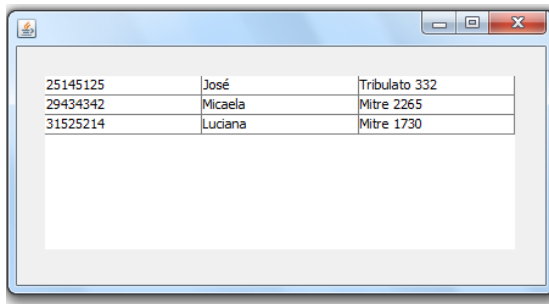
```

1  table = new JTable();
2  ...
3  DefaultTableModel model = new DefaultTableModel();
4  model.addColumn("DNI");
5  model.addColumn("Nombre");
6  model.addColumn("Domicilio");
7
8  model.addRow(new String[] { "25145125", "José", "Tribulato 332" });
9  model.addRow(new String[] { "29434342", "Micaela", "Mitre 2265" });
10 model.addRow(new String[] { "31525214", "Luciana", "Mitre 1730" });
11
12 table.setModel(model);

```

Tablas

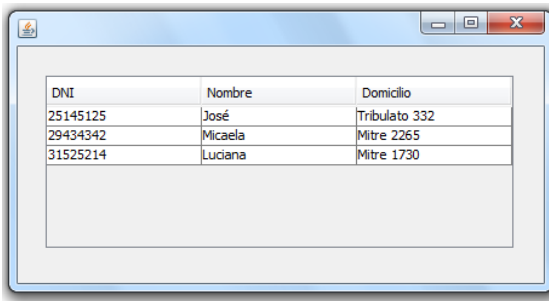
- Sin embargo, la visualización estándar no muestra las columnas del modelo de datos:



25145125	José	Tribulato 332
29434342	Micaela	Mitre 2265
31525214	Luciana	Mitre 1730

Tablas

- Para ver las columnas se debe ubicar la tabla dentro de un **JScrollPane**, con la opción *scrollable with* de eclipse:



DNI	Nombre	Domicilio
25145125	José	Tribulato 332
29434342	Micaela	Mitre 2265
31525214	Luciana	Mitre 1730

Tablas

- Accediendo al modelo de datos de la tabla se pueden consultar y modificar celdas individuales:

Tablas

- Accediendo al modelo de datos de la tabla se pueden consultar y modificar celdas individuales:

```

1  TableModel model = table.getModel();
2  String s = (String)model.getValueAt(2,2); // fila y columna
3  model.setValueAt("Nuevo dato", 1, 1);

```

Tablas

- Para modificar la visualización de las celdas de la tabla, es necesario crear un **TableCellRenderer** con los atributos personalizados que sean necesarios, y asignarlo a la columna correspondiente.

Tablas

- Para modificar la visualización de las celdas de la tabla, es necesario crear un **TableCellRenderer** con los atributos personalizados que sean necesarios, y asignarlo a la columna correspondiente.
- Por ejemplo, para centrar la primera columna de la tabla:

Tablas

- Para modificar la visualización de las celdas de la tabla, es necesario crear un **TableCellRenderer** con los atributos personalizados que sean necesarios, y asignarlo a la columna correspondiente.
- Por ejemplo, para centrar la primera columna de la tabla:

```

1 DefaultTableCellRenderer dtcr = new DefaultTableCellRenderer();
2 dtcr.setHorizontalAlignment( JLabel.CENTER );
3
4 TableColumn col = table.getColumnModel().getColumn(0);
5 col.setCellRenderer(dtcr);

```

Tablas

- Por medio del **TableCellRenderer** se controlan todos los aspectos relacionados con la visualización de las celdas.

Tablas

- Por medio del **TableCellRenderer** se controlan todos los aspectos relacionados con la visualización de las celdas.
- Por ejemplo, para cambiar el color:

Tablas

- Por medio del **TableCellRenderer** se controlan todos los aspectos relacionados con la visualización de las celdas.
- Por ejemplo, para cambiar el color:

```

1  DefaultTableCellRenderer dtcr = new DefaultTableCellRenderer();
2  dtcr.setHorizontalAlignment( JLabel.CENTER );
3  dtcr.setBackground( Color.RED );
4  dtcr.setForeground( Color.WHITE );
5
6  TableColumn col = table.getColumnModel().getColumn(0);
7  col.setCellRenderer(dtcr);

```

Tablas

- El acceso al modelo de las columnas permite modificar otros atributos de las columnas. Por ejemplo, para fijar el ancho de las columnas podemos hacer:

Tablas

- El acceso al modelo de las columnas permite modificar otros atributos de las columnas. Por ejemplo, para fijar el ancho de las columnas podemos hacer:

```

1  table.setAutoResizeMode( JTable.AUTO-RESIZE-OFF );
2
3  table.getColumnModel().getColumn(0).setPreferredWidth(40);
4  table.getColumnModel().getColumn(1).setPreferredWidth(150);
5  table.getColumnModel().getColumn(2).setPreferredWidth(200);

```

Tablas

- Podemos agregar nuevas filas por código si el modelo de datos es un **DefaultTableModel**.

Tablas

- Podemos agregar nuevas filas por código si el modelo de datos es un **DefaultTableModel**.

```

1  DefaultTableModel tm = (DefaultTableModel) table.getModel();
2  tm.addRow( ... );

```

Tablas

- Podemos agregar nuevas filas por código si el modelo de datos es un **DefaultTableModel**.

```

1  DefaultTableModel tm = (DefaultTableModel) table.getModel();
2  tm.addRow( ... );

```

- Si el usuario editó la tabla, accediendo a su modelo de datos se pueden obtener los valores ingresados:

Tablas

- Podemos agregar nuevas filas por código si el modelo de datos es un **DefaultTableModel**.

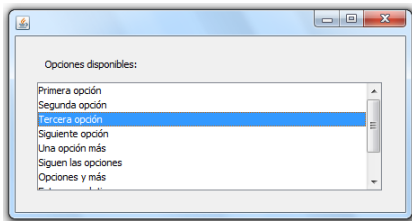
```
1 DefaultTableModel tm = (DefaultTableModel) table.getModel();
2 tm.addRow( ... );
```

- Si el usuario editó la tabla, accediendo a su modelo de datos se pueden obtener los valores ingresados:

```
1 TableModel tm = table.getModel();
2 System.out.println( tm.getValueAt(2, 2) );
```

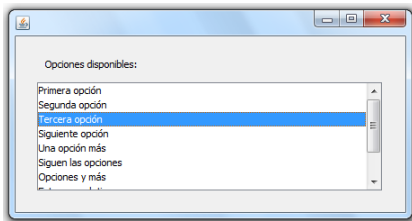
Otras opciones para listas de datos

- Las tablas no son siempre la mejor alternativa para visualizar listas de datos. Si hay una única columna y la selección es importante, se puede usar una **JList**:



Otras opciones para listas de datos

- Las tablas no son siempre la mejor alternativa para visualizar listas de datos. Si hay una única columna y la selección es importante, se puede usar una **JList**:



- Para que se muestre la barra de desplazamiento, se debe rodear la lista con un **JScrollPane**.

Apertura de nuevos forms

- Es habitual que se deba **abrir un nuevo form** desde el form principal o desde otros forms.

Apertura de nuevos forms

- Es habitual que se deba **abrir un nuevo form** desde el form principal o desde otros forms.
- Se deben generar nuevos **JFrames** dentro del proyecto, y se crean con el constructor adecuado (que puede recibir parámetros definidos por el programador).

Apertura de nuevos forms

- Es habitual que se deba **abrir un nuevo form** desde el form principal o desde otros forms.
- Se deben generar nuevos **JFrames** dentro del proyecto, y se crean con el constructor adecuado (que puede recibir parámetros definidos por el programador).

```

1  OtroFrame otro = new OtroFrame();
2  otro.setVisible(true);

```

Apertura de nuevos forms

- Es habitual que se deba **abrir un nuevo form** desde el form principal o desde otros forms.
- Se deben generar nuevos **JFrames** dentro del proyecto, y se crean con el constructor adecuado (que puede recibir parámetros definidos por el programador).

```

1  OtroFrame otro = new OtroFrame();
2  otro.setVisible(true);

```

- La comunicación entre los frames se puede dar por medio de métodos públicos del frame secundario, o bien pasando una referencia al frame llamador.

Apertura de diálogos

- Un **diálogo** (también llamado **form modal**) es una ventana que no permite continuar hasta que no sea cerrada.

Apertura de diálogos

- Un **diálogo** (también llamado **form modal**) es una ventana que no permite continuar hasta que no sea cerrada.
- Se deben generar nuevos **JDialogs** dentro del proyecto, llamando al constructor que recibe el padre y un **boolean** que indica si el diálogo es modal.

Apertura de diálogos

- Un **diálogo** (también llamado **form modal**) es una ventana que no permite continuar hasta que no sea cerrada.
- Se deben generar nuevos **JDialogs** dentro del proyecto, llamando al constructor que recibe el padre y un **boolean** que indica si el diálogo es modal.

```

1  public class DialogoTest extends JDialog
2  {
3      public DialogoTest(JFrame parent)
4      {
5          super(parent, true); // true = modal
6          this.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);
7          ...
8      }
9  }
```

Cuadros de diálogo

- La clase **JOptionPane** permite mostrar cuadros de diálogo con formatos pre-especificados.

Cuadros de diálogo

- La clase **JOptionPane** permite mostrar cuadros de diálogo con formatos pre-especificados.
- Permite generar mensajes y realizar consultas sencillas, sin agregar una clase por cada tipo de mensaje.

Cuadros de diálogo

- La clase **JOptionPane** permite mostrar cuadros de diálogo con formatos pre-especificados.
- Permite generar mensajes y realizar consultas sencillas, sin agregar una clase por cada tipo de mensaje.

```

1  JOptionPane.showMessageDialog(frame,
2      "El archivo se escribió correctamente.");
3
4  JOptionPane.showMessageDialog(frame,
5      "El archivo no se escribió correctamente.",
6      "Problemas", JOptionPane.WARNING_MESSAGE);
7
8  JOptionPane.showMessageDialog(frame, "Error! No se escribió el archivo.",
9      "Error", JOptionPane.ERROR_MESSAGE);
    
```

Cuadros de diálogo

- Se puede especificar una consulta con hasta tres opciones, del siguiente modo:

Cuadros de diálogo

- Se puede especificar una consulta con hasta tres opciones, del siguiente modo:

```

1  Object[] options = {"Sí, por favor", "No, gracias", "Cancelar!" };
2
3  int n = JOptionPane.showOptionDialog(frame,
4      "Se produjo un problema! Desea continuar?",
5      "Problemas",
6      JOptionPane.YES_NO_CANCEL_OPTION,
7      JOptionPane.QUESTION_MESSAGE,
8      null, // Icono
9      options, options[2]); // Valor inicial

```

Ventana principal

- Es habitual que la ventana principal contenga un **menú principal** y una **barra de herramientas**.

Ventana principal

- Es habitual que la ventana principal contenga un **menú principal** y una **barra de herramientas**.
- Se genera el menú agregando al frame la siguiente jerarquía:

Ventana principal

- Es habitual que la ventana principal contenga un **menú principal** y una **barra de herramientas**.
- Se genera el menú agregando al frame la siguiente jerarquía:
 1. El menú principal está dado por un **JMenuBar**.

Ventana principal

- Es habitual que la ventana principal contenga un **menú principal** y una **barra de herramientas**.
- Se genera el menú agregando al frame la siguiente jerarquía:
 1. El menú principal está dado por un **JMenuBar**.
 2. Cada submenú es un **JMenu**.

Ventana principal

- Es habitual que la ventana principal contenga un **menú principal** y una **barra de herramientas**.
- Se genera el menú agregando al frame la siguiente jerarquía:
 1. El menú principal está dado por un **JMenuBar**.
 2. Cada submenú es un **JMenu**.
 3. Dentro de un submenú, cada opción es un **JMenuItem**.

Ventana principal

- Es habitual que la ventana principal contenga un **menú principal** y una **barra de herramientas**.
- Se genera el menú agregando al frame la siguiente jerarquía:
 1. El menú principal está dado por un **JMenuBar**.
 2. Cada submenú es un **JMenu**.
 3. Dentro de un submenú, cada opción es un **JMenuItem**.
- Un **JMenuItem** se comporta del mismo modo que un **JButton**, y tiene asociado un **ActionListener** con el evento **action** → **performed**.

Ventana principal

- Las **opciones chequeables** dentro de los submenús se generan agregando **JCheckBoxMenuItem**, y seteando la propiedad **selected**.

Ventana principal

- Las **opciones chequeables** dentro de los submenús se generan agregando **JCheckBoxMenuItem**, y seteando la propiedad **selected**.
- Para generar una barra de herramientas, se debe agregar un **JToolBar** y darle las dimensiones adecuadas.

Ventana principal

- Las **opciones chequeables** dentro de los submenús se generan agregando **JCheckBoxMenuItems**, y seteando la propiedad **selected**.
- Para generar una barra de herramientas, se debe agregar un **JToolBar** y darle las dimensiones adecuadas.
 1. Se agregan los componentes necesarios al **JToolBar**, especialmente **JButtons**.

Ventana principal

- Las **opciones chequeables** dentro de los submenús se generan agregando **JCheckBoxMenuItems**, y seteando la propiedad **selected**.
- Para generar una barra de herramientas, se debe agregar un **JToolBar** y darle las dimensiones adecuadas.
 1. Se agregan los componentes necesarios al **JToolBar**, especialmente **JButtons**.
 2. Si no queremos que el usuario modifique la ubicación de la barra de herramientas, se debe desactivar la propiedad **floatable**.

Consejos generales

- Es una buena práctica agregar **tooltip texts** a los controles, especialmente a los botones de la barra de herramientas.

Consejos generales

- Es una buena práctica agregar **tooltip texts** a los controles, especialmente a los botones de la barra de herramientas.
- Los controles se organizan habitualmente por medio de **JPanels**.

Consejos generales

- Es una buena práctica agregar **tooltip texts** a los controles, especialmente a los botones de la barra de herramientas.
- Los controles se organizan habitualmente por medio de **JPanels**.
 1. La propiedad **border** cuenta con muchas opciones para manejar la visualización de los paneles.

Consejos generales

- Es una buena práctica agregar **tooltip texts** a los controles, especialmente a los botones de la barra de herramientas.
- Los controles se organizan habitualmente por medio de **JPanels**.
 1. La propiedad **border** cuenta con muchas opciones para manejar la visualización de los paneles.
 2. Una opción muy utilizada es usar un **TitledBorder**, que genera un nombre en la parte superior.

Consejos generales

- Es una buena práctica agregar **tooltip texts** a los controles, especialmente a los botones de la barra de herramientas.
- Los controles se organizan habitualmente por medio de **JPanels**.
 1. La propiedad **border** cuenta con muchas opciones para manejar la visualización de los paneles.
 2. Una opción muy utilizada es usar un **TitledBorder**, que genera un nombre en la parte superior.
 3. Cada panel se maneja como un frame y es importante ubicar un **AbsoluteLayout** si queremos organizar los controles por medio de coordenadas absolutas.